Reporte y Explicación del Código C++ (Gestor de Ordenamiento - POO)

En este programa adaptamos el algoritmo Merge Sort (Ordenamiento por Mezcla) al paradigma de Programación Orientada a Objetos. Se creó la clase GestorOrdenamiento para encapsular tanto los contenedores de datos (vectores de enteros y caracteres) como la lógica de ordenamiento recursivo y la interacción con el usuario mediante un menú en consola.

Código Completo:

```
#include <iostream>
#include <vector>

// --- CLASE QUE IMPLEMENTA LA POO ---
class GestorOrdenamiento {
private:
    std::vector<int> vecEnteros;
    std::vector<char> vecCaracteres;

    // Merge Sort Privado para Enteros
    void mergeEnteros(std::vector<int>& arr, int izq, int medio, int der) {
        int n1 = medio - izq + 1;
        int n2 = der - medio;
        std::vector<int> L(n1), R(n2);
        for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
        for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];

        int i = 0, j = 0, k = izq;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }

    void mergeSortEnterosHelper(std::vector<int>& arr, int izq, int der) {
        if (izq < der) {
            int medio = izq + (der - izq) / 2;
            mergeSortEnterosHelper(arr, izq, medio);
            mergeSortEnterosHelper(arr, medio + 1, der);
            mergeEnteros(arr, izq, medio, der);
        }
    }

    // Merge Sort Privado para Caracteres
    void mergeCaracteres(std::vector<char>& arr, int izq, int medio, int der) {
        int n1 = medio - izq + 1;
        int n2 = der - medio;
        std::vector<char> L(n1), R(n2);
        for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
        for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];

        int i = 0, j = 0, k = izq;
```

```
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }

    void mergeSortCaracteresHelper(std::vector<char>& arr, int izq, int der) {
        if (izq < der) {
            int medio = izq + (der - izq) / 2;
            mergeSortCaracteresHelper(arr, izq, medio);
            mergeSortCaracteresHelper(arr, medio + 1, der);
            mergeCaracteres(arr, izq, medio, der);
        }
    }

public:
    // Método para ingresar y ordenar Enteros
    void gestionarEnteros() {
        int n;
        std::cout << "Cantidad de números enteros a ingresar: ";
        std::cin >> n;
        if (n <= 0) return;

        vecEnteros.resize(n);
        std::cout << "Ingrese los " << n << " números:\n";
        for (int i = 0; i < n; i++) {
            std::cout << "Elemento [" << i + 1 << "]: ";
            std::cin >> vecEnteros[i];
        }

        mergeSortEnterosHelper(vecEnteros, 0, n - 1);

        std::cout << "\n[POO] Vector de enteros ordenado:\n[ ";
        for (int i = 0; i < n; i++) {
            std::cout << vecEnteros[i] << " ";
        }
        std::cout << "]\n";
    }

    // Método para ingresar y ordenar Caracteres
    void gestionarCaracteres() {
        int n;
        std::cout << "Cantidad de caracteres a ingresar: ";
        std::cin >> n;
        if (n <= 0) return;

        vecCaracteres.resize(n);
        std::cout << "Ingrese los " << n << " caracteres:\n";
        for (int i = 0; i < n; i++) {
            std::cout << "Caracter [" << i + 1 << "]: ";
            std::cin >> vecCaracteres[i];
        }

        mergeSortCaracteresHelper(vecCaracteres, 0, n - 1);

        std::cout << "\n[POO] Vector de caracteres ordenado:\n[ ";
```

```
        for (int i = 0; i < n; i++) {
            std::cout << vecCaracteres[i] << " ";
        }
        std::cout << "]\n";
    }
};

// --- FUNCIÓN PRINCIPAL (MAIN) ---
int main() {
    GestorOrdenamiento gestor;
    int opcion = 0;

    do {
        std::cout << "\n=======================================\n";
        std::cout << "   MENÚ INTERACTIVO - MERGE SORT (POO)  \n";
        std::cout << "=======================================\n";
        std::cout << "1. Ordenar enteros\n";
        std::cout << "2. Ordenar caracteres\n";
        std::cout << "3. Salir\n";
        std::cout << "Seleccione una opción: ";

        if (!(std::cin >> opcion)) break;

        switch (opcion) {
            case 1:
                gestor.gestionarEnteros();
                break;
            case 2:
                gestor.gestionarCaracteres();
                break;
            case 3:
                std::cout << "Saliendo del programa POO... ¡Hasta luego!\n";
                break;
            default:
                std::cout << "Opción no válida. Intente de nuevo.\n";
        }

    } while (opcion != 3);

    return 0;
}
```

Explicación del Código:

1. Encapsulamiento de Datos y Métodos Privados (private:)

Los contenedores vecEnteros y vecCaracteres están protegidos dentro de la clase. Además, las funciones auxiliares de ordenamiento (mergeEnteros, mergeSortEnterosHelper, mergeCaracteres y mergeSortCaracteresHelper) son privadas, ocultando los detalles de implementación al exterior.

2. Algoritmo Merge Sort (Divide y Vencerás)

Las funciones helper dividen recursivamente los vectores a la mitad calculando el punto medio. Luego, los métodos de mezcla (merge) crean subvectores temporales L (izquierda) y R (derecha) para comparar e ir ordenando los elementos de menor a mayor.

3. Métodos Públicos de Interfaz (gestionarEnteros y gestionarCaracteres)
Son la puerta de acceso al usuario. Capturan el número de elementos, leen la entrada desde la consola, invocan los ordenamientos privados y finalmente despliegan los resultados ya ordenados.

4. Función Principal (main)
Instancia un único objeto gestor de la clase GestorOrdenamiento y ejecuta un menú interactivo basado en un ciclo do-while y switch-case para permitir múltiples pruebas de ordenamiento antes de salir del programa.